

Week12_예습과제

트랜스포머

- 기존의 순환 신경망과 같은 순차적 방식이 아닌 병렬로 입력 시퀀스를 처리
- 셀프 어텐션 기반: 모델이 재귀나 합성곱 연산 없이 입력 토큰 간의 관계 직접 모델링함.
→ 대용량 데이터셋에서 매우 효율적

오토 인코딩

- 랜덤하게 문장의 일부를 빈칸 토큰으로 만들고 해당 빈칸의 단어를 예측하는 작업 수행
- 빈칸 주변 양옆 토큰을 예측에 참고 → **양방향** 구조 : **인코더**



그림 7.1 트랜스포머 구조

- A는 **높고** 를 예측하기 위해서 양 옆의 토큰들을 참고하는 양방향 구조 ⇒ **인코더**
- B는 **높고** 를 예측하기 위해 한 쪽 방향의 토큰만을 참고하는 단방향 구조 ⇒ **디코더**

표 7.1 트랜스포머 모델 구조

모델	학습구조	학습방법	학습 방향성
BERT	인코더	오토 인코딩	양방향
GPT	디코더	자기 회귀	단방향
BART	인코더+디코더	오토 인코딩+자기 회귀	양방향+단방향
ELECTRA	인코더+판별기	오토 인코딩+대체 토큰 탐지	양방향
T5	인코더+디코더	오토 인코딩+자기 회귀+다양한 자연어 처리 작업을 학습	양방향

Transformer

- 다양한 자연어 처리 분야에서 많은 성과를 내는 딥러닝 모델
- 어텐션 메커니즘(Attention Mechanism)만을 사용하여 시퀀스 임베딩을 표현

어텐션 메커니즘

- input 또는 output 데이터에서 sequence distance에 무관하게 서로 간의 상관도를 모델링
- 인코더와 디코더 간의 상호작용으로 입력 시퀀스의 중요한 부분에 포커싱해 문맥을 이해하고 예측을 수행한다.
- 인코더: 입력 시퀀스를 임베딩하여 고차원 벡터로 변환
- 디코더: 인코더 출력을 입력으로 받아 출력 시퀀스를 생성
- 어텐션 메커니즘은 인코더와 디코더 단어 사이의 상관관계를 계산해 중요한 정보에 집중

트랜스포머 모델 구조

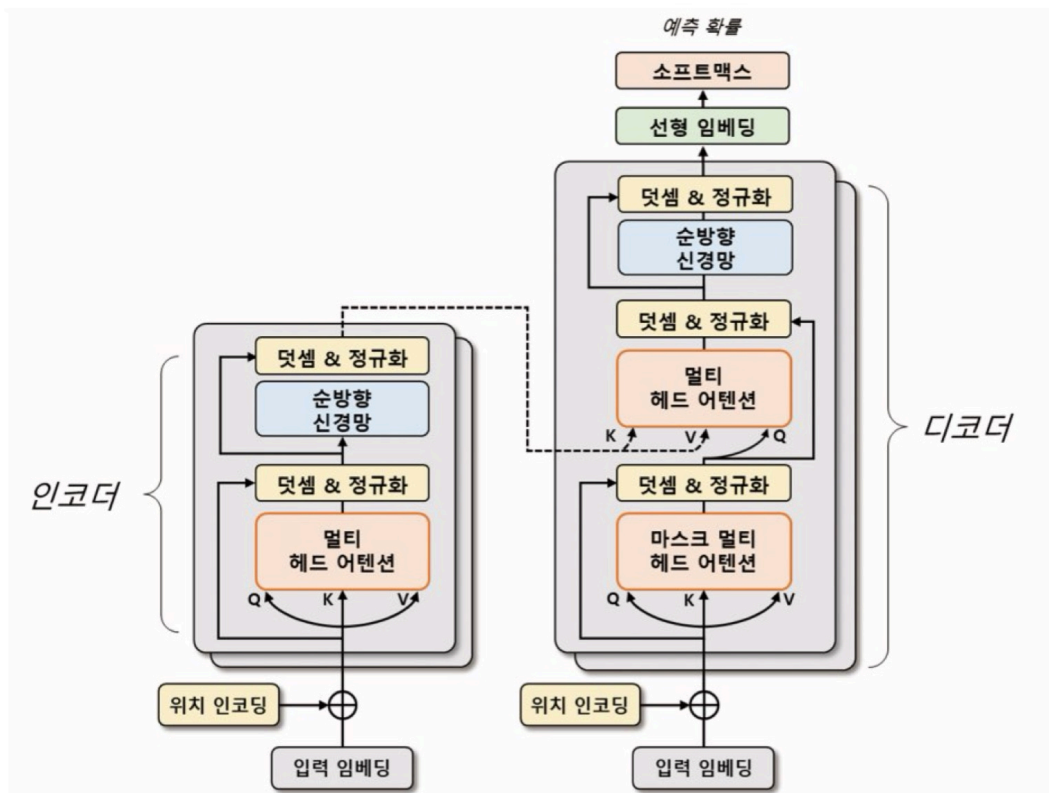


그림 7.2 트랜스포머 모델 구조(Q: 쿼리, K: 키, V: 값)

- 인코더와 디코더 두 부분으로 구성 (encoder-decoder 구조)
- 각각 N개의 **트랜스포머 블록**(Tranformer Block)으로 구성됨
- 블록은 **멀티 헤드 어텐션**과 **순방향 신경망**으로 이루어짐



멀티 헤드 어텐션이란?

- 입력에 쿼리(Query), 키(Key), 값(Value) 벡터를 정의해 입력 시퀀스들의 관계를 셀프 어텐션하는 벡터 표현 방법
- 이 과정에서 쿼리와 각 키의 유사도를 계산하고 해당 유사도를 가중치로 사용하여 값 벡터를 합산한다.
- 계산된 어텐션 행렬은 입력 시퀀스 각 단어의 임베딩 벡터를 대체
- 하나의 attention function보다 쿼리,키,값을 중간중간 매핑하여 각 다른 값들을 입력으로 하는 여러 개의 attention function들을 만드는 방법으로, 더 효율적이다.

⇒ 입력 시퀀스 단어 사이의 상호작용을 고려해 임베딩 벡터를 갱신

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

어텐션

- Query(쿼리): 물어보는 주체
- Key(키): 쿼리에게 물어봄을 당하는 주체
- Value(값): 데이터의 값

$$\text{Attention}(Q, K, V) = \text{softmax}(\sqrt{d_k}QK^T\sqrt{d_k}V)$$

순방향 신경망

- 위 과정에서 산출된 임베딩 벡터를 고도화

- 여러 개의 선형 계층으로 구성되어 입력 벡터에 **가중치**를 곱하고 편향을 더하며 활성화 함수를 적용
- 가중치는 입력 시퀀스의 각 단어의 의미를 잘 파악하는 방식으로 갱신된다.

트랜스포머에서는 입력 시퀀스 데이터를 새로 생성하는 **타겟 데이터**와 참조하는 **소스 데이터**로 나눠 처리한다. 인코더는 소스 데이터를 **위치 인코딩**된 입력 임베딩으로 표현해 트랜스포머 블록의 출력 벡터를 생성한다. 디코더는 **마스크 멀티 헤드 어텐션**을 사용해 타겟 데이터를 순차적으로 생성한다.

입력 임베딩과 위치 인코딩

- 트랜스포머 모델에서 입력 시퀀스의 각 단어는 임베딩 처리되어 벡터 형태로 변환됨.
- 트랜스포머 모델은 RNN과 달리 입력 시퀀스를 병렬 구조로 처리하기 때문에 시퀀스 정보가 손실됨 → 위치 정보를 임베딩 벡터에 추가하여 순서 정보 반영! → 이를 위해 **위치 인코딩** 방식이 도입됨.

위치 인코딩

- 입력 시퀀스의 순서 정보를 모델에 전달하는 방법으로, 각 단어의 위치 정보를 나타내는 벡터를 더하여 임베딩 벡터에 위치 정보를 반영한다.
- 위치 인코딩 벡터

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

▼ 위치 인코딩에 대해 더 알아보기

위치 인코딩이 필요한 이유

- 트랜스포머 모델은 병렬적으로 입력 시퀀스를 처리하며 연산의 효율성이라는 이득을 얻지만, 언어 모델에 있어서 "어순"은 중요하기에 손실된 시퀀스 정보를 보충하기 위해 위치 인코딩이 필요하다.

위치 정보를 어떻게 줄 것인가?

1. 데이터의 순서마다 0~1 사이의 label을 붙이는 방법
한계: input의 총 크기를 알 수 없음
2. 각 time-step마다 선형적 숫자를 할당하는 방법

한계: 숫자가 매우 커질 수 있다.

조건

1. 각 단어의 위치마다 하나의 유일한 encoding 값을 출력해야함.
2. 서로 다른 길이의 문장에서 두 time-step간 거리가 일정해야 함.
3. 모델에 대한 일반화가 가능해야 함.
4. 매번 같은 값이 나와야 함.

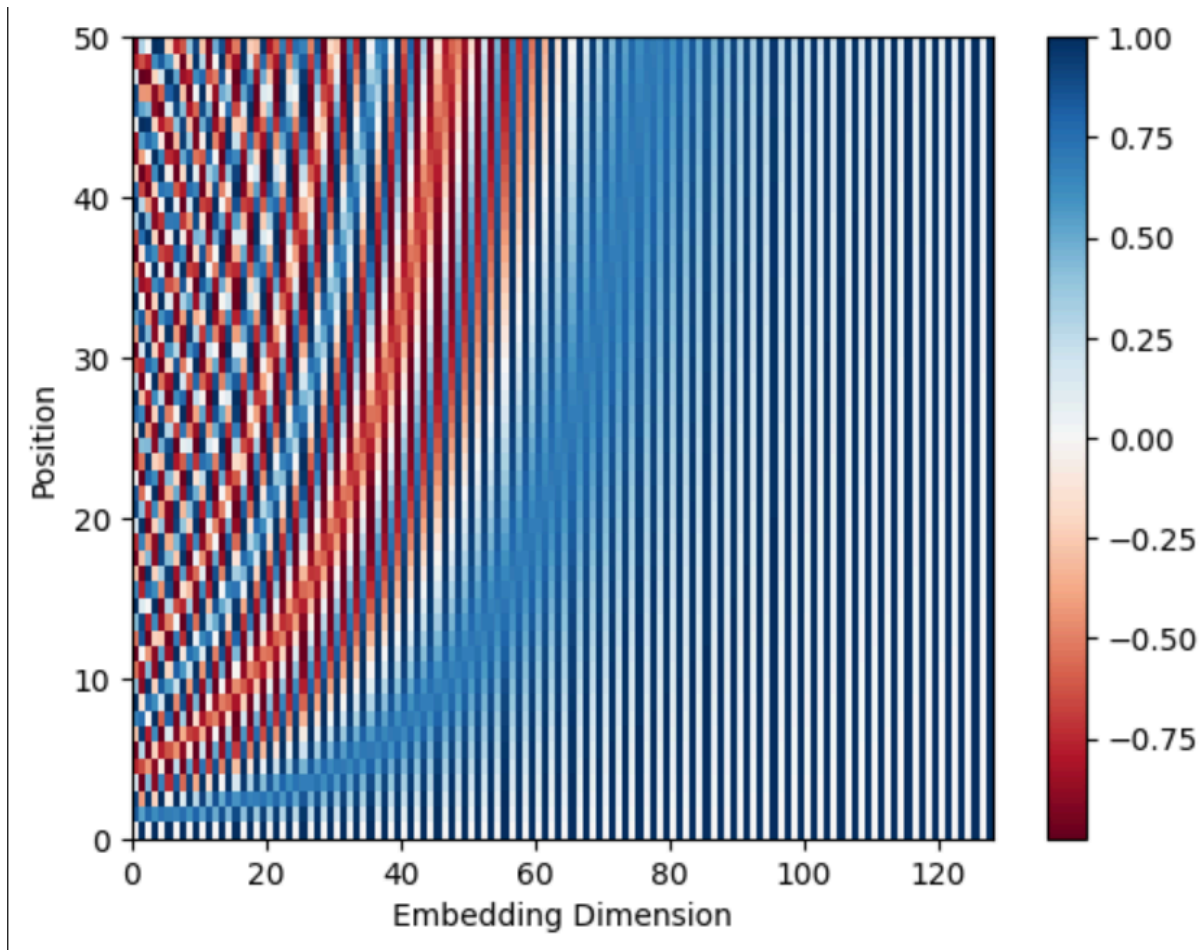
트랜스포머에서 사용된 인코딩

- 위의 조건들을 모두 만족함.

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

- pos : 문장 내에서 단어의 위치 (0번째 단어, 1번째 단어...)
- i : 임베딩 벡터 내의 차원 인덱스
- d_{model} : 임베딩 차원의 전체 크기 (예: 512)
- $1/10000^{2i/d_{model}}$: 각도 정보로 변환하기 위한 스케일링 인자로 각 위치에 대한 주기적 신호 생성
- 짝수 차원에서는 $\sin$ 을, 홀수 차원에서는 $\cos$ 함수를 사용 → 각 위치마다 고유한 벡터값
⇒ 삼각함수의 덧셈 정리 성질에 의해 선형 변환으로 표현 가능하여 모델이 상대적 위치 개념을 수학적으로 쉽게 계산하고 학습 가능하게 함.



위치 인코딩 계산 과정

- 임베딩 차원(d_{model})과 최대 시퀀스(max_len)를 입력 받아 입력 시퀀스의 위치마다 위치 인코딩을 계산한다.

특수 토큰

- 트랜스포머가 문장 표현에 사용하는 단어 토큰 이외의 토큰
- 입력 시퀀스의 시작과 끝을 나타내거나 마스킹(Masking) 영역으로 사용된다.
- 마스킹: 일부 입력을 무시하게 함

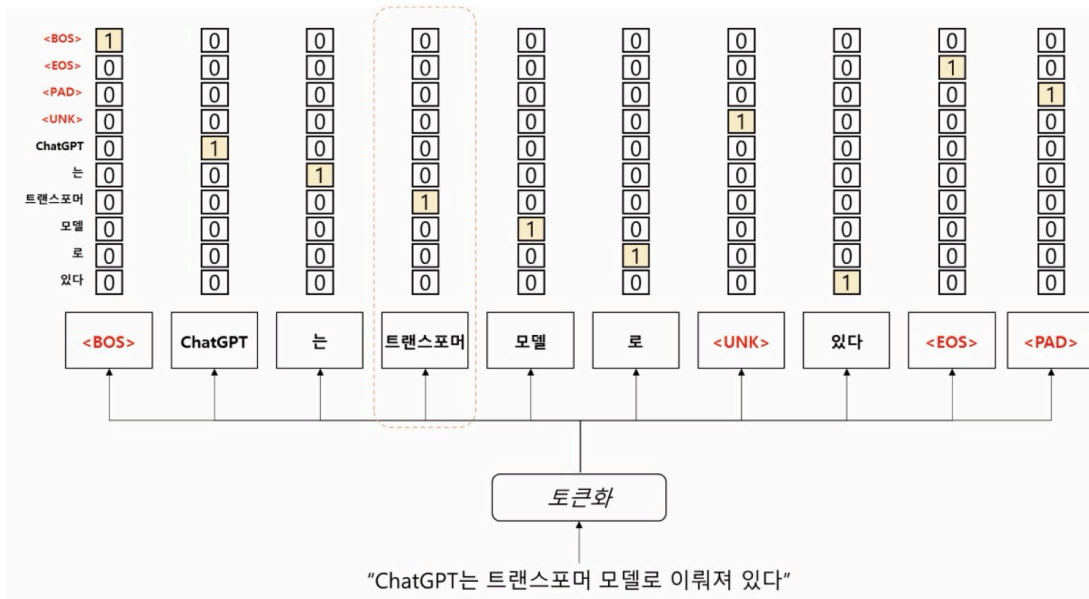


그림 7.3 특수 토큰을 활용한 문장 토큰 배열

- 7.3에서 사용한 토큰들
 - BOS(Beginning of Sentence) : 문장의 시작을 나타내는 토큰
 - EOS(End of Sentence) : 문장의 종료를 나타내는 토큰
 - UNK(Unknown) : 어휘 사전에 없는 토큰으로 모델이 이전에 본 적 없는 단어를 처리할 때 사용된다.
 - PAD(Padding): 모든 문장을 일정한 길이로 맞추기 위해 짧은 문장의 빈 공간을 채울 때 사용되는 토큰

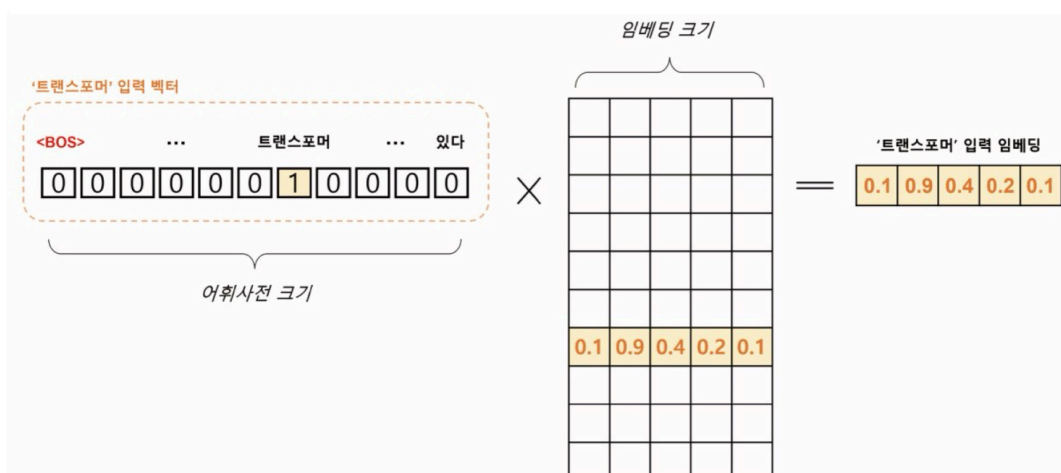


그림 7.4 입력 임베딩 산출 과정

- 7.3에서 생성된 문장 토큰 배열을 어휘 사전에 등장하는 위치에 원-핫 인코딩으로 표현한다.

- V : 어휘 사전의 크기
- d : 입력 임베딩 차원
- 트랜스포머의 원-핫 벡터는 $[1, V]$ 크기를 갖고 임베딩 행렬 $[V, d]$ 에 의해 $[1, d]$ 크기의 벡터로 변환됨.
- N 개의 문장이 최대 S 개의 토큰 길이를 가질 때, $[N, S, V]$ 크기의 원-핫 벡터 텐서는 $[N, S, d]$ 크기의 임베딩 텐서로 변환되어 입력으로 사용됨.

트랜스포머 인코더

트랜스포머 인코더 구조 및 과정

- 여러 개의 계층으로 이루어져 있음
- 입력 시퀀스를 받아 여러 계층으로 구성된 인코더 계층을 거쳐 연산 수행
- 각 인코더 계층은 멀티 헤드 어텐션과 순방향 신경망으로 구성되며, 입력 데이터에 대한 정보를 추출하고 다음 계층으로 전달함.
- 인코더 계층에서 위치 정보 반영을 위해 위치 임베딩 벡터를 입력 벡터에 더해줌
- 산출된 인코더 계층의 출력은 디코더 계층으로 전달.

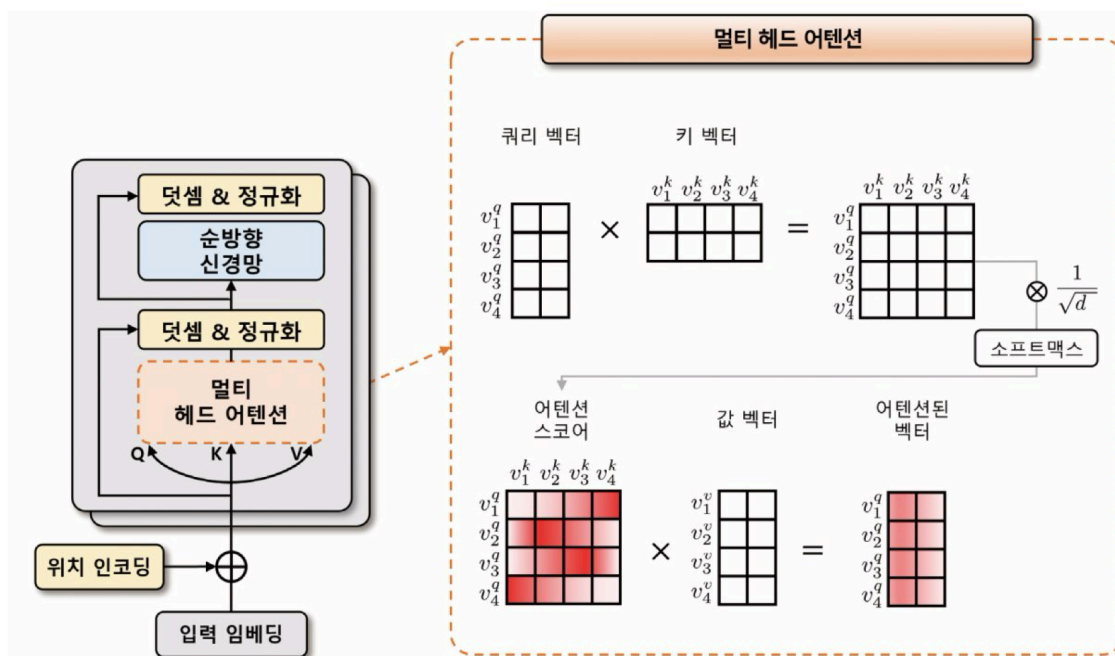


그림 7.5 트랜스포머 인코더 블록 연산 과정

연산 과정

1. 입력으로 위치 인코딩이 적용된 소스 데이터의 입력 임베딩을 입력 받는다.
 2. 멀티 헤드 어텐션 단계에서 $[N, S, d]$ 의 입력 텐서 차원을 선형 변환을 통해 쿼리(Q), 키(K), 값(V)의 3개의 임베딩 벡터를 생성한다.
 - a. **쿼리 벡터(v^q)**: 현재 시점에서 참고하고자 하는 정보의 위치를 나타내는 벡터
 - b. **키 벡터(v^k)**: 입력 시퀀스에서 쿼리 벡터를 제외한 탐색되는 벡터
 - c. **값 벡터(v^v)**: 쿼리 벡터와 키 벡터로 생성된 어텐션 스코어를 얼마나 반영할지 설정하는 가중치 역할의 벡터
- 초기에는 이 셋이 비슷한 값을 가지지만, 모델 학습을 통해 의도한 의미의 값을 갖게 된다.

수식 7.2 어텐션 스코어 계산식

$$score(v^q, v^k) = \text{softmax}\left(\frac{(v^q)^T \cdot v^k}{\sqrt{d}}\right)$$

- 어텐션 스코어: 쿼리, 키 벡터를 내적하고 벡터차원 제곱근으로 나눠 보정한 뒤, 소프트맥스 함수를 통해 확률적으로 재표현함.
- 셀프 어텐션 스코어: 어텐션 스코어를 값 벡터와 내적해 생성한 벡터
- **멀티 헤드**는 이러한 셀프 어텐션을 여러 번 수행해 여러 개의 독립적인 헤드를 만들고 각각의 결과를 임베딩 차원 축으로 다시 병합한다.
- **덧셈&정규화**는 멀티 헤드 어텐션 통과 이전값과 출력값 텐서를 더해 기울기 소실을 완화한다.
- 이 값에 임베딩 차원 축으로 정규화하는 **계층 정규화**를 적용한다.
- **순방향 신경망**: 두 가지 방법 적용 가능
 - 인공 신경망: ex) 선형 임베딩과 ReLU
 - 1차원 합성곱

트랜스포머 디코더

- 위치 인코딩이 적용된 타겟 데이터의 입력 임베딩을 입력 받음.
- **마스크 멀티 헤드 어텐션** 모듈을 사용해 어텐션 스코어 맵 계산 시, 셀프 어텐션에서 현재 위치 이전의 단어들만 참조하여 인과성을 보장함.

연산 과정

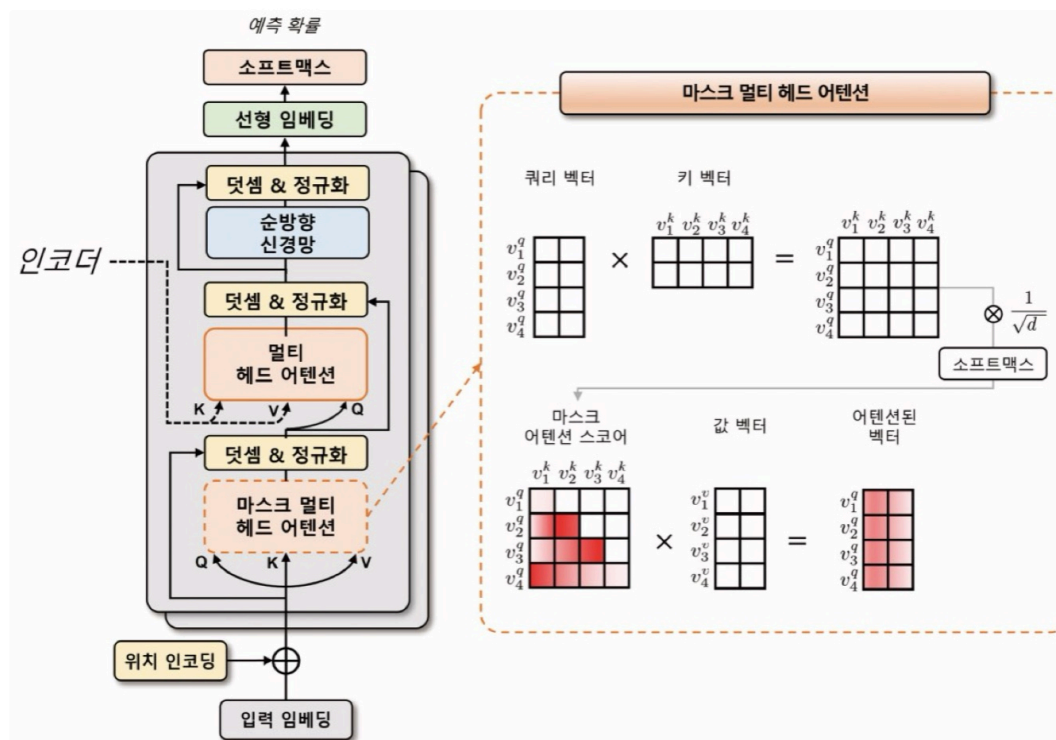


그림 7.6 트랜스포머 디코더 블록 연산 과정

- 멀티 헤드 어텐션에서 타깃 데이터: 쿼리 벡터, 키와 값 벡터: 인코더의 소스 데이터
 - 쿼리 벡터: 타깃 데이터의 위치 정보를 포함한 입력 임베딩 + 위치 인코딩
- 쿼리, 키, 값 벡터를 이용해 어텐션 스코어 맵 계산
- 소프트맥스 함수를 적용해 어텐션 가중치 계산
- 어텐션 가중치와 값 벡터를 가중합하여 출력 벡터를 얻음

표 7.2 인과성을 고려한 디코더 입력과 출력 예시

추론 순서	디코더 입력	디코더 출력
1	[BOS], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]
2	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]
3	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD], [PAD]
4	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD], [PAD]
5	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD]
6	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD]
7	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [EOS], [PAD]

모델 실습

Multi30k

- 자연어 처리를 위한 대규모 다국어 데이터셋
- 영어-독일어 병렬 말뭉치로 30,000개의 데이터 제공
- 토치 데이터와 토치 텍스트 라이브러리
 - 토치 데이터 라이브러리: 대규모 데이터셋을 다루기 쉽게 데이터를 불러오고 변환 및 배치하는 유연하고 간단한 API 제공
 - 토치 텍스트 라이브러리: 파이토치를 위한 텍스트 라이브러리
- 포르타락커(portalocker) 라이브러리: 파이썬에서 여러 프로세스 간 파일 동시수정.읽기 방지하는 파일락 관리를 위한 라이브러리

실습 코드 참조

GPT

- 2018년 OpenAI에서 발표한 트랜스포머 기반 언어 모델

- ELMo(Embeddings from Language Model)와 같이 대규모 말뭉치로 사전 학습된 모델로 다양한 다운스트림 작업에서 우수한 성능을 보임
- GPT 모델은 트랜스포머의 **디코더**를 여러 층 쌓아 만든 언어 모델
 - 인코더는 입력 문장의 특징을 추출하는 데 초점을 두고 있다면, 디코더는 입력 문장과 출력 문장 간의 관계를 이해하고 출력 문장을 생성하는 데 초점
 - 디코더를 쌓아 만든 모델은 자연어 생성과 같은 언어 모델링 작업에서 더 높은 성능 발휘

GPT의 종류

거의 동일한 트랜스포머 구조를 사용하며, 버전이 갱신될수록 더 많은 트랜스포머 계층과 데이터 활용해 학습됨.

- GPT-1 (2018년): 첫 번째 모델로, 1억 2천만 개의 매개변수가 존재
- GPT-2 (2019년): 1의 성능을 크게 능가하여 약 15억 개의 매개변수
- GPT-3 (2020년): 1,750억 개의 매개변수를 찾는 초거대 모델로 매우 뛰어난 문장 생성 능력
- GPT-4 (2023년): 이미지 데이터까지 처리

많은 양의 매개변수를 줄이기 위해 도입한 학습 방법 : **RLHF**(Reinforcement Learning from Human Feedback)

RLHF

- 인간의 피드백을 이용하는 강화 학습 방법
- 인간이 모델이 생성한 결과물을 평가하고, 보상을 주는 방식으로 모델을 학습시킨다.
 - 사용자와의 대화를 통해 지속적으로 학습하며 발전

GPT-1

GPT-1의 구조

- 트랜스포머 구조를 바탕으로 한 **단방향(Unidirectional)** 언어 모델
 - 단방향 언어 모델은 텍스트 생성 시 현재 위치 기준 이전 단어들에 대한 정보만을 참고해 다음 단어를 예측한다.
 - 계산량이 줄기 때문에 학습 속도 빠르고, 모델의 크기 작음
- 트랜스포머 구조 중 **디코더 부분만**을 사용

- 디코더의 두 번째 멀티헤드 어텐션은 사용X
- 12개의 헤드를 가진 디코더를 12층 쌓은 구조

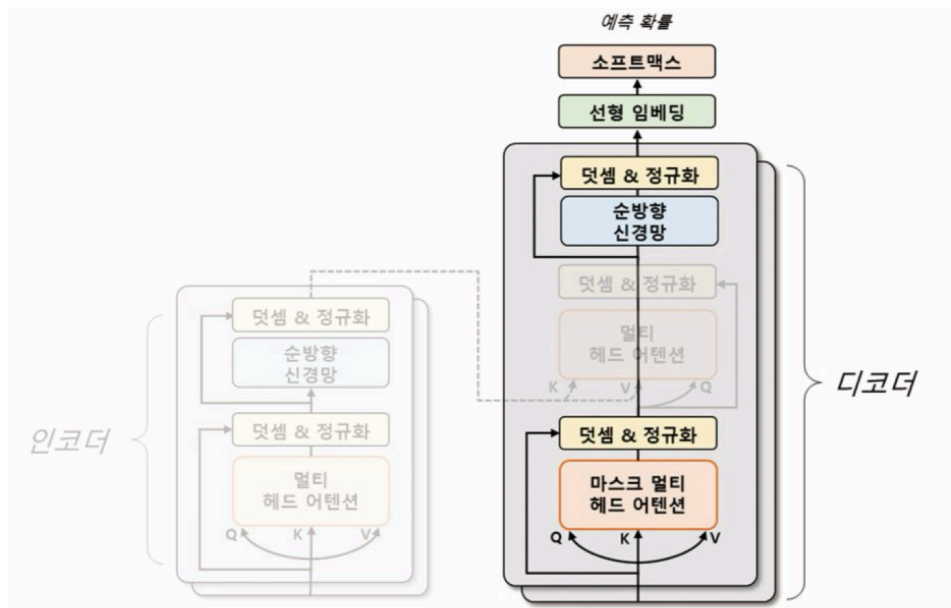


그림 7.7 트랜스포머와 GPT-1 모델 비교

GPT-1의 학습 구조

- 입력 문장을 임의의 위치까지 보여주고 뒤에 이어지는 문장을 모델이 예측하게 함
 - 레이블이 필요하지 않는 비지도 학습
 - 컴퓨팅 자원만 충분하다면 제약 없이 학습 가능

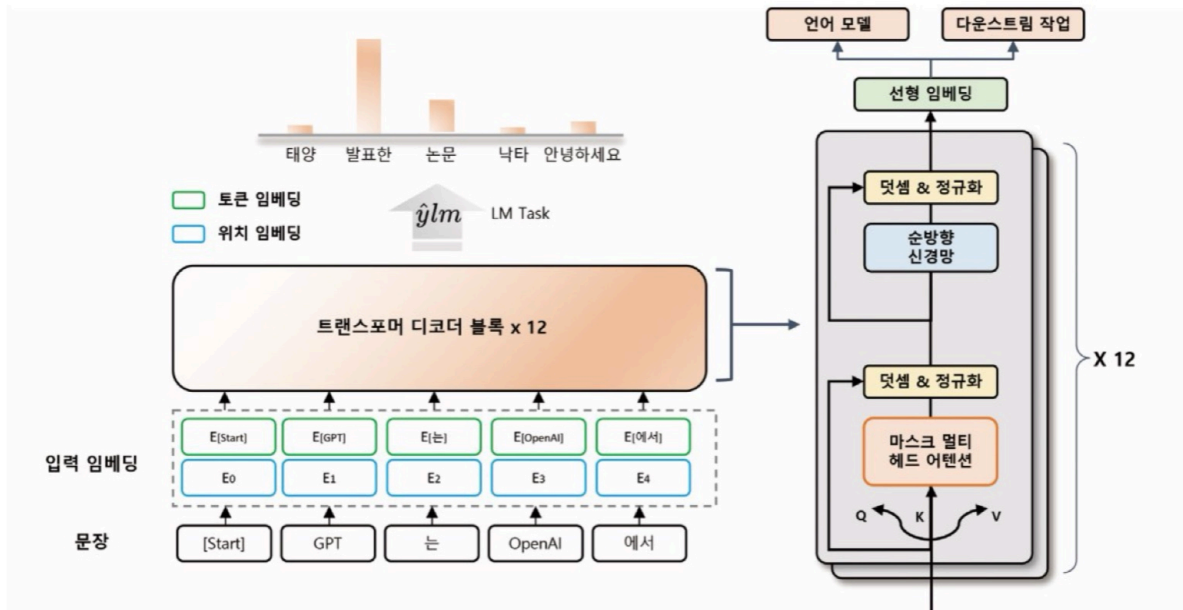


그림 7.8 GPT-1의 사전 학습 작업 구조

GPT-1의 학습 과정

- 입력 문장의 일부만 보고 다음 토큰을 예측하는 언어 모델을 통해 사전 학습
- 입력 문장을 일정한 길이의 블록으로 나눈다.
- 각 블록에서는 블록 내의 첫 번째 토큰부터 순서대로 다음 토큰을 예측한다.
- 계산: 입력 임베딩은 토큰 임베딩과 위치 임베딩을 이용해 계산된다.
 - 토큰 임베딩: 각 토큰을 고정된 차원의 벡터로 매핑
 - 위치 임베딩: 입력 문장에서 각 토큰의 위치 정보를 나타내는 벡터를 매핑

GPT-1의 다운스트림

- **보조 학습:** 모델이 주어진 작업 외에도 다른 작은 부가적인 학습 작업을 동시 수행하는 것
 - 보조 학습한 정보를 주요 작업에 전달하여 성능을 향상
 - GPT-1은 미세 조정을 위한 다운스트림 작업에서도 각 작업에 따라 입출력 구조를 바꾸지 않고 보조 학습을 수행한다.

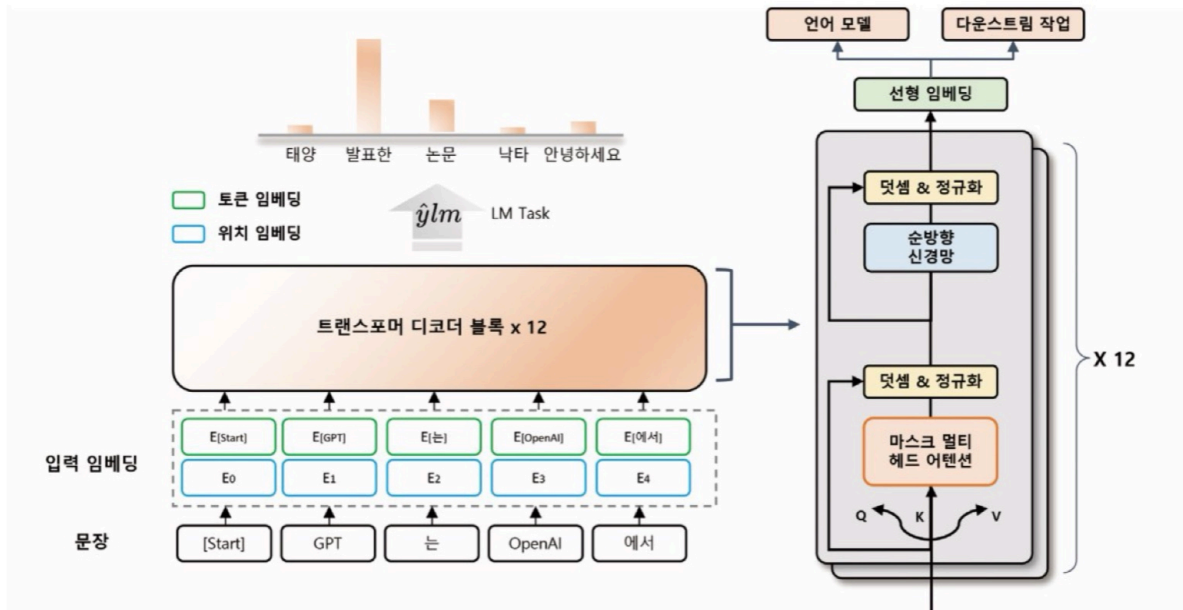


그림 7.8 GPT-1의 사전 학습 작업 구조

- 다운스트림에서는 일반적인 언어 모델 학습 시 사용되는 손실 함수 외에 모델 세밀 조정을 위해 **추가 손실 함수가 필요하다**.
→ 두 개의 손실함수로 최적화하면 보조 학습을 통해 언어 모델 개선과 다운스트림 작업 목표 달성을 위해 필요한 정보를 학습할 수 있다.
- 다운스트림 작업에 사용되는 **특수 토큰**
 - <start> 토큰 : 문장의 시작
 - <delim> 토큰 : 두 문장의 경계
 - <extract> 토큰 : 문장의 마지막

GPT-2

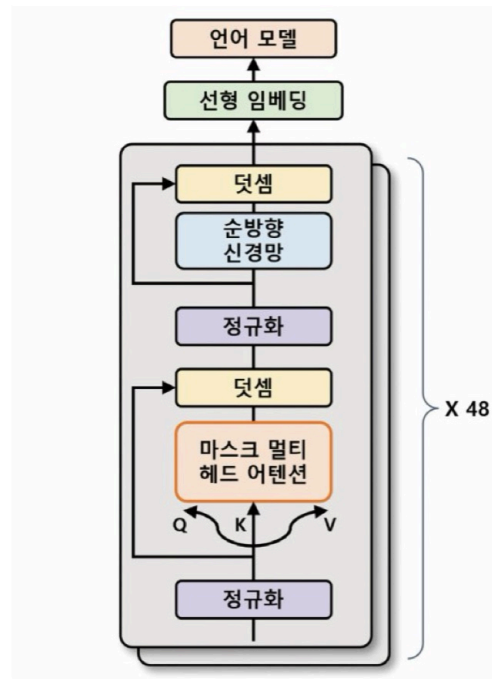


그림 7.10 GPT-2의 모델 구조

GPT-2의 특징

- 더욱 자연스러운 문장 생성 능력
- 활용 분야의 다양화
- 제로-샷 학습
 - 별도의 미세 조정 없이 다운스트림 작업에서도 사용할 수 있게 사전 학습 과정에서 특정한 형식의 데이터 입력 → 학습 과정으로 배우지 않은 작업에도 사용 가능

GPT-3

- 2와 거의 동일한 모델 구조, 모델 매개변수에 약간의 변화
- 멀티 헤드 어텐션 과정에서 연산량 감소를 위해 희소 어텐션과 일반 어텐션을 섞어 사용함.



그림 7.11 GPT-3의 다운스트림 작업 프롬프트 예시

특징

- 새로운 도메인에 대해 높은 일반화
- 단점
 - 매우 느리고 큰 비용
 - 사전 학습 데이터에 기반 → 데이터 편향, 오류 발생
 - 인간적인 이해력과 상식적 추론 능력 부족
 - 편견, 혐오 표현의 가능성

GPT-1 vs GPT-2 vs GPT-3

	GPT-1	GPT-2	GPT-3
입력 문장의 최대 길이	512	1,024	2,048
디코더 층의 개수	12개	48개	96개
매개변수의 수	1,200만 개	15억 개	
학습 데이터 크기	4.5GB	40GB	45TB

GPT 3.5 (InstructGPT)

- GPT-3의 모델 구조와 유사

- 새로운 학습법인 **RLHF** 도입

RLHF

- 데이터셋에서 가져온 임의의 프롬프트에 대해 사람이 직접 적절한 답을 작성
- 이 답을 모델이 생성한 문장과 함께 평가 "**지도 미세 조정(Supervised Fine-Tuning, SFT)**"
 - 모델이 생성한 문장이 자연스럽다면? → 보상O
 - 그렇지 않다면? → 보상X

RLHF에서는 주어진 프롬프트에 대해 GPT-3 언어 모델이 답변을 **생성**하고, 이를 이용해 사람의 피드백을 기반으로 한 학습된 보상 모델을 **평가**한다.

⇒ 이 과정을 반복해 정교한 모델 생성 가능

- 성능이 높지만 일부 입력에 대해 **환각 현상**을 보임.
 - 환각 현상: 모델이 인간과 같은 추론과 판단 능력을 갖추지 못해 오류 답변을 제시하는 현상

GPT-4

- 텍스트 뿐 아니라 이미지 데이터 인식도 가능한 **멀티모달** 모델
 - 멀티모달: 다양한 종류의 데이터를 동시에 처리하는 기술
- GPT-3.5보다 더 많은 데이터 처리와 더 향상된 성능이 특징

GPT 실습

```
# 문장 생성을 위한 GPT-2 모델 구조
from transformers import GPT2LMHeadModel

model = GPT2LMHeadModel.from_pretrained(pretrained_model_name_or_path="gpt2")

for main_name, main_module in model.named_children():
    print(main_name)
    for sub_name, sub_module in main_module.named_children():
        print(" L", sub_name)
```

```

for ssub_name, ssub_module in sub_module.named_children():
    print(" |  L", ssub_name)
    for sssub_name, sssub_module in ssub_module.named_children():
        print(" | |  L", sssub_name)

```

```

transformer
├─ wte
├─ wpe
├─ drop
├─ h
│   └─ 0
│       ├── ln_1
│       ├── attn
│       ├── ln_2
│       └─ mlp
│   └─ 1
│       ├── ln_1
│       ├── attn
│       ├── ln_2
│       └─ mlp
│   └─ 2
│       ├── ln_1
│       ├── attn
│       ├── ln_2
│       └─ mlp
│   └─ 3
│       ├── ln_1
│       ├── attn
│       ├── ln_2
│       └─ mlp
│   └─ 4
│       ├── ln_1
│       ├── attn
│       ├── ln_2
│       └─ mlp
│   └─ 5
│       ├── ln_1
│       ├── attn
│       ├── ln_2
│       └─ mlp

```

```

├─ 6
│   ├── ln_1
│   ├── attn
│   ├── ln_2
│   └─ mlp
├─ 7
│   ├── ln_1
│   ├── attn
│   ├── ln_2
│   └─ mlp
├─ 8
│   ├── ln_1
│   ├── attn
│   ├── ln_2
│   └─ mlp
├─ 9
│   ├── ln_1
│   ├── attn
│   ├── ln_2
│   └─ mlp
├─ 10
│   ├── ln_1
│   ├── attn
│   ├── ln_2
│   └─ mlp
├─ 11
│   ├── ln_1
│   ├── attn
│   ├── ln_2
│   └─ mlp
└─ ln_f
    └─ lm_head

```

- 트랜스포머 라이브러리의 `GPT2LMHead` 클래스의 `from_pretrained` 메서드로 사전 학습된 GPT-2 모델 불러오기
- `pretrained_model_name_or_path` : 불러오려는 사전 학습된 모델을 의미하는 매개변수

모델 구조 살펴보기

- 12개의 디코더 계층을 사용하는 간소화 모델인 gpt2
 - gpt2-xl : 48개의 디코더 계층 사용하는 GPT-2 모델
- `wte` : 단어 토큰 임베딩
- `wpe` : 단어 위치 임베딩
- `drop` : 드롭아웃
- `h` : 트랜스포머 디코더 계층
- `lm_head` : 선형 임베딩 및 언어 모델

파이프라인 클래스

```

# GPT-2를 이용한 문장 생성
from transformers import pipeline

generator = pipeline(task="text-generation", model="gpt2") # 텍스트 생성 작업
outputs = generator(
    text_inputs="Machine learning is",
    max_length=20, # 생성될 문장의 최대 토큰 수 제한
    num_return_sequences=3, # 생성할 텍스트 시퀀스의 수
    pad_token_id=generator.tokenizer.eos_token_id # pad_token_id : 모델의
    자유 생성 여부

)
print(outputs)

```

- **pipeline** : 문장 분류 및 생성, 토큰 분류 등 다양한 작업에 대한 전처리, 모델 아키텍처, 후처리를 각각 처리할 수 있는 함수
- 입력된 작업에 모델로 적합한 파이프라인을 구축함

데이터셋 불러오기

```

# CoLA 데이터셋 불러오기
import torch
from torchtext.datasets import CoLA
from transformers import AutoTokenizer
from torch.utils.data import DataLoader

```

```

def collator(batch, tokenizer, device):
    source, labels, texts = zip(*batch)
    tokenized = tokenizer(
        texts,
        padding="longest",
        truncation=True,
        return_tensors="pt"
    )
    inputs_ids = tokenized["input_ids"].to(device)
    attention_mask = tokenized["attention_mask"].to(device)

```

```

labels = torch.tensor(labels, dtype=torch.long).to(device)
return input_ids, attention_mask, labels
train_data = list(CoLA(split="train"))
valid_data = list(CoLA(split="dev"))
test_data = list(CoLA(split="test"))

tokenizer = AutoTokenizer.from_pretrained("gpt2")
tokenizer.pad_token = tokenizer.eos_token

epochs=3
batch_size=16
device="cuda" if torch.cuda.is_available() else "cpu"

train_dataloader = DataLoader(
    train_data,
    batch_size=batch_size,
    collate_fn=lambda x: collator(x, tokenizer, device),
    shuffle=True,
)
valid_dataloader=DataLoader(
    valid_data, batch_size=batch_size, collate_fn=lambda x:collator(x, tokeni
zer, device)
)
test_dataloader=DataLoader(
    test_data, batch_size=batch_size, collate_fn=lambda x:collator(x, tokeniz
er, device)
)
print("Train Dataset Length: ", len(train_data))
print("Valid Dataset Length: ", len(valid_data))
print("Test Dataset Length: ", len(test_data))

```

```

Train Dataset Length: 8550
Valid Dataset Length: 526
Test Dataset Length: 515

```

colab 실습코드 참조